

Components and Props

Components: Functional vs. Class, Creating and Rendering
Components, Passing & Accessing Props, Difference between
Props and State,
this.props.children and React Fragments, Component Lifecycle
(Overview), Rendering
Components

Components Vs Props

1. Components

- **Definition:**

Components are **independent, reusable pieces of UI** in a React application.

- **Purpose:**

They define **how a part of the UI looks and behaves**.

- **Types:**

- **Functional Components**
- **Class Components**

Example

// A simple component

```
function Welcome() {  
  return <h1>Hello, User!</h1>;  
}
```

// Using the component

```
export default function App() {  
  return <Welcome />;  
}
```

```
function Welcome() { ... }
```

- This defines a **React functional component** named **Welcome**.
- In React, a **component** is just a **JavaScript function** that returns **JSX** (which looks like HTML).
- Here it returns:

```
<h1>Hello, User!</h1>
```

- That means whenever we use **<Welcome />**, React will render an **<h1>** heading with the text **"Hello, User!"** on the screen.

So this component acts like a **custom HTML tag**.

export default function App() { ... }

- This defines another component called **App**.
- It's marked as export default, which means it's the **main component** that React will render first (typically inside index.js).

Inside it, we have:

return <Welcome />;

This means the **App component uses the Welcome component** inside it — just like adding a tag `<Welcome />`. So when App renders:

1. **It finds <Welcome />**
2. **Calls the Welcome function**
3. **Gets <h1>Hello, User!</h1>**
4. **Displays it on the screen**

Props (Properties)

- Props are **inputs** to components — like **arguments passed to a function**. They allow you to **pass data from parent to child components**.
- **Purpose:** To make components **dynamic and reusable**.

Example:

```
// Child component

function Welcome(props) {
  return <h1>Hello, {props.name}!</h1>;
}

// Parent component passing a prop
export default function App() {
  return <Welcome name="abc" />;
}
```

Component = A function (e.g.,
Welcome())

Prop = A parameter (e.g.,
name="abc")

Function component

In React, a **function component** is simply a **JavaScript function** that **returns JSX (HTML-like code)** to describe how the UI should look.

Earlier, React used **class components** for UI + state.

But now, with **React Hooks**, function components can do everything classes can — with **simpler, cleaner code**.

```
// App.js
```

```
function Welcome() {
```

```
  return <h1>Hello, User!</h1>;
```

```
}
```

```
export default Welcome;
```

function Welcome()

- This defines a **React function component** named Welcome.
- It's just like a normal JavaScript function.

return <h1>Hello, User!</h1>;

- Instead of returning a number or text, React functions return **JSX** (HTML-like syntax).
- JSX tells React what should appear on the screen.

export default Welcome;

- This allows you to use <Welcome /> in another file, like App.js.

When React renders:

It calls Welcome() → gets <h1>Hello, User!</h1> → displays it on the web page.

```
import React, { useState } from "react";

function Counter() {
  // Declare a state variable 'count' and a function 'setCount' to update it
  const [count, setCount] = useState(0);

  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  );
}

export default Counter;
```

Function Component with State (using Hooks)

Functions use **Hooks** (like `useState`) to store and update data inside components.

```
import { useState } from "react";
```

- Brings in the useState hook from React.

```
const [count, setCount] = useState(0);
```

- Declares a state variable count with an initial value of 0.
- setCount is the function used to update it.

```
<h2>Count: {count}</h2>
```

- Displays the current value of count.

```
<button onClick={() => setCount(count + 1)}>Increment</button>
```

- When the button is clicked, setCount updates the value of count.
- React re-renders the component with the new count.

Each click updates the UI instantly — no page reload needed.

useState is a **React Hook**.

- It lets function components have their **own state**.
- Normally, functions are stateless, but React Hooks changed that.

Now, the Counter function **remembers** the value of number even after re-rendering.

Class component

A **Class Component** is another way of writing components in React. It uses **JavaScript ES6 classes** and is capable of using **state** and **lifecycle methods**.

Before React Hooks were introduced, **Class Components were the only way** to use state and lifecycle methods in React.

```
import React, { Component } from "react";

class Welcome extends Component {

  render() {

    return <h1>Hello, {this.props.name}!</h1>;

  }

}

export default Welcome;
```

```
import React from "react";  
import Welcome from "../Welcome";  
function App() {  
  return (  
    <div>  
      { /* Passing name prop to Welcome */}  
      <Welcome name="Shruti" />  
      <Welcome name="Rohan" />  
    </div>  
  );  
}  
export default App;
```

Line	Meaning
class Welcome extends Component	Creates a new class named Welcome that inherits from <u>React's</u> built-in Component class
render()	Class components must have a render method . This method returns the JSX UI
this.props.name	Used to access props in class components
export default Welcome	Allows this component to be imported elsewhere

Class Component with State (Important)-This shows how **class components handle state**, which is their main feature.

```
import React, { Component } from "react";

class Counter extends Component {
  constructor() {
    super(); // Always required in class components
    this.state = {
      count: 0
    };
  }
}
```

```
increment = () => {  
  this.setState({ count: this.state.count + 1 });  
};  
  
render() {  
  return (  
    <div>  
      <h2>Count: {this.state.count}</h2>  
      <button onClick={this.increment}>Increment</button>  
    </div>  
  );  
}  
}  
  
export default Counter;
```

Part	Meaning
constructor()	Initializes the component
<u>this.state = { count: 0 }</u>	Defines initial state
increment()	Custom method to update state
<u>this.setState()</u>	Updates the state and re-renders UI
<u>this.state.count</u>	Access state value
render()	Must return JSX



Feature	Function Component	Class Component
Syntax	function Counter() { ... }	class Counter extends Component { ... }
State	useState hook	this.state & this.setState
Lifecycle	useEffect hook	componentDidMount, componentDidUpdate, componentWillUnmount
Event Handler	Simple function or inline	Arrow function
Code Length	Short, concise	Longer, more boilerplate
Modern Usage	Preferred	Older style, less used now
JSX Return	Directly inside function	Inside render() method

TABLE 4-2: Functions vs. Classes

FUNCTION COMPONENTS	CLASS COMPONENTS
Accepts props as arguments and returns a React element	Extends <code>React.Component</code>
No render method	Requires a render method
No internal state (can be simulated using hooks)	Has internal state
Can use hooks	Cannot use hooks
Cannot use lifecycle methods (can be simulated using hooks)	Can use lifecycle methods

```
import React, { useState, useEffect } from "react";
```

```
function Example() {
```

```
  const [count, setCount] = useState(0);
```

```
  useEffect(() => {
```

```
    // This runs after every render
```

```
    console.log(`You clicked ${count} times`);
```

```
  });
```

```
  return (
```

```
    <div>
```

```
      <p>You clicked {count} times</p>
```

```
      <button onClick={() => setCount(count + 1)}>Click me</button>
```

```
    </div>
```

```
  );
```

```
}
```

include:

- Fetching data from an API.
- Starting a timer.
- Updating the browser document title.
- Setting up a subscription to receive updates.
- Adding an event listener to the browser.

React provides **useEffect** to manage these operations.

Creating Components

In React, a **component** is a **reusable piece of UI**. You can create components in **two ways**:

Function Component (Modern way)

```
// Greeting.js
```

```
import React from "react";
```

```
function Greeting(props) {
```

```
  return <h1>Hello, {props.name}!</h1>;
```

```
}
```

```
export default Greeting;
```

B. Class Component (Older way)

```
import React, { Component } from "react";

class GreetingClass extends Component {

  render() {

    return <h1>Hello, {this.props.name}!</h1>;

  }

}

export default GreetingClass;
```

Rendering Components

Once you've created a component, you **render** it in your main file (usually index.js or App.js).

Example: Using the Function Component

// App.js

```
import React from "react";

import Greeting from "../Greeting";

function App() {

  return (

    <div>

      <Greeting name="abc" />

      <Greeting name="xyz" />

    </div>

  );

}

export default App;
```

Greeting.js

```
import React from "react";

function Greeting(props) {
  return (
    <h2>Hello, {props.name}!</h2>
  );
}

export default Greeting;
```

Passing & Accessing Props

Passing Props

You **pass props** by adding attributes to a component when you use it.

```
function App() {  
  
  return (  
  
    <div>  
  
      <Greeting name="abc" age={22} />  
  
      <Greeting name="xyz" age={25} />  
  
    </div>  
  
  );  
  
}
```

- Here, <Greeting /> is the child component
- name and age are **props** passed to it

Accessing Props in Function Components

```
function Greeting(props) {  
  return (  
    <h1>  
      Hello, {props.name}! You are {props.age} years old.  
    </h1>  
  );  
}  
  
export default Greeting;
```

Using Destructuring (Simpler)

```
function Greeting({ name, age }) {  
  return <h1>Hello, {name}! You are {age} years old.</h1>;  
}
```

- { name, age } → extracts the props directly
- Makes code cleaner and shorter

Props

```
function Greeting(props) {  
  return <h1>Hello, {props.name}!</h1>;  
}  
  
function App() {  
  return <Greeting name="Shruti" />;  
}
```

State

```
import React, { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  );
}
```

Feature	Props	State
Purpose	Pass data from parent → child	Store dynamic data inside component
Mutability	Read-only	Can be changed
Managed by	Parent component	Local component
Trigger re-render	Only if parent passes new props	Always triggers re-render
Function analogy	Function arguments	Local variable that can change
Usage example	<Greeting name="abc" />	<u>const</u> [count, <u>setCount</u>] = <u>useState</u> (0)

this.props.children

What is it?

- this.props.children allows a component to **render whatever is placed inside its opening and closing tags.**
- Think of it like a **placeholder** for child content.
- Works in **both class and function components.**

```
function Card(props) {  
  return <div className="card">{props.children}</div>;  
}  
  
function App() {  
  return (  
    <Card>  
      <h2>Title</h2>  
      <p>This is some content inside the card.</p>  
    </Card>  
  );  
}
```

Explanation:

1. `<Card>` wraps some JSX (`<h2>` and `<p>`)
2. Inside `Card`, `props.children` contains everything inside the tags
3. `Card` renders:

```
<div class="card">
```

```
  <h2>Title</h2>
```

```
  <p>This is some content inside the card.</p>
```

```
</div>
```

```
import React, { Component } from "react";  
class Box extends Component {  
  render() {  
    return (  
      <div style={{ border: "2px solid blue", padding: "10px" }}>  
        {this.props.children}  
      </div>  
    );  
  }  
}  
  
export default Box;
```

Usage in App.js:

```
import React from "react";
```

```
import Box from "../Box";
```

```
function App() {
```

```
  return (
```

```
    <Box>
```

```
      <h1>Hello Shruti!</h1>
```

```
      <p>This is some text inside the box.</p>
```

```
    </Box>
```

```
  );
```

```
}
```

```
export default App;
```

React Fragments

- React Fragments let you **group multiple elements without adding extra nodes to the DOM.**
- Normally, JSX requires a single parent element. Fragments solve this without adding extra `<div>`s.

Syntax Examples: Using `<React.Fragment>`

```
function List() {  
  
  return (  
  
    <React.Fragment>  
  
      <li>Item 1</li>  
  
      <li>Item 2</li>  
  
      <li>Item 3</li>  
  
    </React.Fragment>  
  
  );  
}
```

Feature	Purpose	Example
props.children	Pass content between opening/closing tags to child component	<Card> {content} </Card>
React Fragment	Group multiple elements without extra DOM node	12 </>

What are React Children?

In React, components can contain other components. These nested components are called **children** and can be accessed using `props.children`.

Utility	Purpose
React.Children.map	Loop through children & return new elements
React.Children.forEach	Loop through children but no return
React.Children.count	Count number of children
React.Children.only	Verify only one child exists
React.Children.toArray	Convert children to an array
React.isValidElement	Check if a value is a valid React element
React.cloneElement	Clone and modify child elements

Example 1: React.Children.map

```
import React from "react";

function ListWrapper(props) {
  return (
    <div>
      {React.Children.map(props.children, child => (
        <div style={{ border: "1px solid gray" }}>{child}</div>
      ))}
    </div>
  );
}
```

```
export default function App() {  
  return (  
    <ListWrapper>  
      <p>Item 1</p>  
      <p>Item 2</p>  
    </ListWrapper>  
  );  
}
```

Example 2: React.Children.count

```
import React from "react";

function CountChildren(props) {
  return <p>Total children: {React.Children.count(props.children)}</p>;
}

export default function App() {
  return (
    <CountChildren>
      <p>A</p>
      <p>B</p>
      <p>C</p>
    </CountChildren>
  );
}
```

Example 3: cloneElement

```
import React from "react";
```

```
function NavBar(props) {
```

```
  return (
```

```
    <div>
```

```
      {React.Children.map(props.children, child =>
```

```
        React.cloneElement(child, { onClick: props.onClick })
```

```
      )}
```

```
    </div>
```

```
  ); }
```

```
function NavItem(props) {
```

```
  return <button onClick={props.onClick}>Click Me</button>;
```

```
}
```

cloneElement makes **copies of child components** and **adds props to each one**.

Example 4: React.isValidElement

```
function ValidateChild(props) {  
  return React.isValidElement(props.child)  
    ? <p>Valid React element!</p>  
    : <p>Not a valid element</p>;  
}  
  
export default function App() {  
  return <ValidateChild child={<h3>Hello</h3>} />;  
}
```

```
import React from "react";

function ValidateChild(props) {
  return React.isValidElement(props.child)
    ? <p>Valid React element!</p>
    : <p>Not a valid element</p>;
}

export default function App() {
  return (
    <div>
```

```
{/* Valid React element */}
```

```
<ValidateChild child={<h3>Hello</h3>} />
```

```
{/* Not a valid React element (string) */}
```

```
<ValidateChild child="I am just a string" />
```

```
{/* Not a valid React element (number) */}
```

```
<ValidateChild child={123} />
```

```
</div>
```

```
);
```

```
}
```

Component Lifecycle (Overview)

Every **React component** goes through a **series of “phases”** from creation to removal. These are called **lifecycle methods**.

Phase	Description	Common Methods
Mounting	Component is being created and added to the DOM	constructor(), render(), componentDidMount()
Updating	Component is re-rendered due to state or props change	render(), componentDidUpdate(prevProps, prevState)
Unmounting	Component is removed from the DOM	componentWillUnmount()

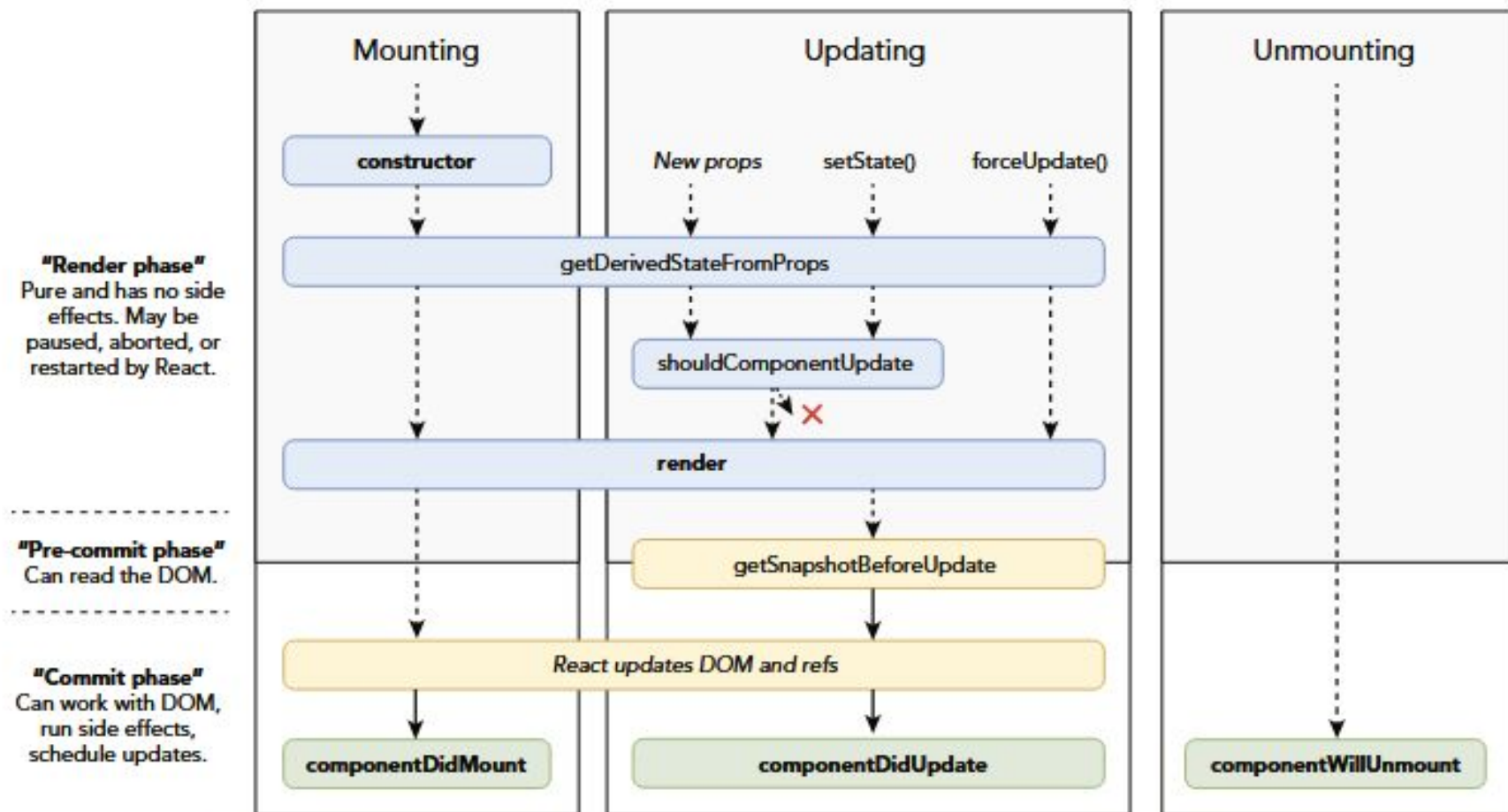


FIGURE 4-5: The component lifecycle

Mounting Stage

Mounting is the phase when a React component is **created and inserted into the DOM**.

During mounting, these methods are called in order:

1. **constructor()**
2. **static getDerivedStateFromProps()**
3. **render()**
4. **componentDidMount()**

Mounting – when the component is added to the DOM

Methods called in order:

1. **constructor()** – Initialize state and bind events
2. **static getDerivedStateFromProps()** – Sync state with props if needed
3. **render()** – Return JSX
4. **componentDidMount()** – Runs after the component is in the DOM; good for API calls, timers, or DOM operations



Example: Showing data from an API when the component loads

constructor()

- Automatically runs when a component instance is created.
- Common tasks:
 - Initialize state
 - Bind event handlers

```
import React, { Component } from "react";  
class MyComponent extends Component {  
  constructor(props) {  
    super(props);  
    this.state = { count: 0 };  
    console.log("Constructor called");  
  }  
  render() {  
    return <h1>Count: {this.state.count}</h1>;  
  }  
}  
export default MyComponent;
```

static getDerivedStateFromProps(props, state)

- Runs **before rendering**, both during mounting and updating.
- Checks if props have changed and updates state accordingly.
- It's **static**, so no `this` keyword.

```
class MyComponent extends Component {  
  constructor(props) {  
    super(props);  
    this.state = { count: 0 };  
  }  
  static getDerivedStateFromProps(props, state) {  
    console.log("getDerivedStateFromProps called");  
    if (props.initialCount !== state.count) {  
      return { count: props.initialCount };  
    }  
    return null; // no state update  
  }  
}
```

```
render() {  
  return <h1>Count: {this.state.count}</h1>;  
}  
  
}  
  
export default function App() {  
  return <MyComponent initialCount={5} />;  
}
```

render()

- Returns the **JSX output** of the component.
- **Required** in class components.
- Called after constructor and `getDerivedStateFromProps`.

```
render() {  
  console.log("Render called");  
  return <h1>Count: {this.state.count}</h1>;  
}
```

componentDidMount()

- Called **after the component is mounted in the DOM.**
- Safe to perform:
 - DOM operations
 - API calls
 - Subscriptions

```
componentDidMount() {  
  console.log("Component mounted!");  
  // Example: fetch data  
  // fetch("https://api.example.com/data").then(...)  
}
```

```
import React, { Component } from "react";  
class MyComponent extends Component {  
  constructor(props) {  
    super(props);  
    this.state = { count: 0 };  
    console.log("Constructor");  
  }  
  
  static getDerivedStateFromProps(props, state) {  
    console.log("getDerivedStateFromProps");  
    return null;  
  }  
}
```

**Complete
example**

```
componentDidMount() {  
  console.log("componentDidMount");  
}
```

```
render() {  
  console.log("Render");  
  return <h1>Count: {this.state.count}</h1>;  
}  
}
```

```
export default function App() {  
  return <MyComponent />;  
}
```

```
import React, { Component } from "react";  
  
class Welcome extends Component {  
  constructor() {  
    super();  
    console.log("Constructor: Component is created");  
  }  
  
  componentDidMount() {  
    console.log("ComponentDidMount: Component added to DOM");  
  }  
}
```

Example

```
render() {  
  console.log("Render: Component is rendering UI");  
  return <h1>Hello, {this.props.name}</h1>;  
}  
}  
  
export default Welcome;
```

Explanation:

1. constructor() → called **first** when component is created
2. render() → called to render **JSX to the DOM**
3. componentDidMount() → called **after component is on the page**, e.g., good for fetching data

Updating Stage

After a component is **mounted**, it can **update** in response to:

- Changes in **state** (using `this.setState`)
- Changes in **props** from a parent component

During updating, the following methods run in **this order**:

1. `static getDerivedStateFromProps`
2. `shouldComponentUpdate`
3. `render`
4. `getSnapshotBeforeUpdate`
5. `componentDidUpdate`

`getDerivedStateFromProps` and `render` are also called in the mounting phase, so the **new lifecycle-specific methods** here are:

`shouldComponentUpdate`, `getSnapshotBeforeUpdate`, `componentDidUpdate`.

Updating – when the component re-renders due to state or props changes

Methods called in order:

1. **static getDerivedStateFromProps()** – Sync state with new props
2. **shouldComponentUpdate()** – Decide if component should re-render (return true/false)
3. **render()** – Update the JSX
4. **getSnapshotBeforeUpdate()** – Capture DOM info before changes (e.g., scroll position)
5. **componentDidUpdate()** – Runs after update; can use snapshot or fetch new data

✓ Example: Updating a counter or filtering a list

shouldComponentUpdate(nextProps, nextState)

- Controls whether the component should **re-render**.
- By default, React **always re-renders** on state or prop change.
- Return **true** → update
- Return **false** → skip update

```
import React, { Component } from "react";

class Counter extends Component {
  constructor(props) {
    super(props);
    this.state = { count: 0 };
  }

  shouldComponentUpdate(nextProps, nextState) {
    // Only update if count is even
    return nextState.count % 2 === 0;
  }

  increment = () => {
    this.setState({ count: this.state.count + 1 });
  };
}
```

```
render() {  
  console.log("Render", this.state.count);  
  return (  
    <div>  
      <h1>Count: {this.state.count}</h1>  
      <button onClick={this.increment}>Increment</button>  
    </div>  
  );  
}  
}  
  
export default Counter;
```

getSnapshotBeforeUpdate(prevProps, prevState)

- Runs **right before the DOM updates**.
- Lets you **capture information from the DOM** before changes.
- Return value from this method is passed as **third argument** to `componentDidUpdate`.

`componentDidUpdate(prevProps, prevState, snapshot)`

- Runs **after the component updates**.
- Useful for:
 - Fetching data based on updated props
 - Performing DOM operations using info from `getSnapshotBeforeUpdate`

```
componentDidUpdate(prevProps, prevState) {  
  console.log("Updated from", prevState.count, "to",  
    this.state.count);  
}
```

shouldComponentUpdate → Optimize performance by preventing unnecessary renders

getSnapshotBeforeUpdate → Capture DOM info **before update**

componentDidUpdate → Act **after update**, e.g., network requests or DOM changes

Updating Phase Example

```
class Counter extends Component {  
  constructor() {  
    super();  
    this.state = { count: 0 };  
  }  
  increment = () => {  
    this.setState({ count: this.state.count + 1 });  
  };  
  componentDidUpdate(prevProps, prevState) {  
    console.log("Previous count:", prevState.count);  
    console.log("Updated count:", this.state.count);  
  }  
}
```

```
render() {  
  return (  
    <div>  
      <p>Count: {this.state.count}</p>  
      <button onClick={this.increment}>Increment</button>  
    </div>  
  );  
}
```

`componentDidUpdate()` runs after state or props change

Helps in performing side-effects after update (e.g., API calls)

What is Unmounting?

- **Unmounting** is the process when a component is **removed from the DOM**.
- Happens when:
 - A component is no longer needed.
 - Conditional rendering removes it.
 - Parent component re-renders without this child.

Unmounting – when the component is removed from the DOM

Method:

- **componentWillUnmount()** – Cleanup (timers, subscriptions, event listeners)



Example: Stop a timer or cancel an API call when component is gone

Lifecycle Method: **componentWillUnmount**

- This is the **only lifecycle method** called during unmounting.
- Invoked **right before the component is removed from the DOM**.
- Purpose: **clean up resources** (timers, network requests, event listeners) to avoid memory leaks.

Unmounting Phase Example

```
componentWillUnmount() {  
  console.log("Component is being removed from DOM");  
  // Useful to clear timers, cancel API calls, remove event listeners  
}
```

Mounting → “Set up everything” (constructor, render, componentDidMount)

Updating → “Something changed” (state/props → render → componentDidUpdate)

Unmounting → “Clean up before leaving”